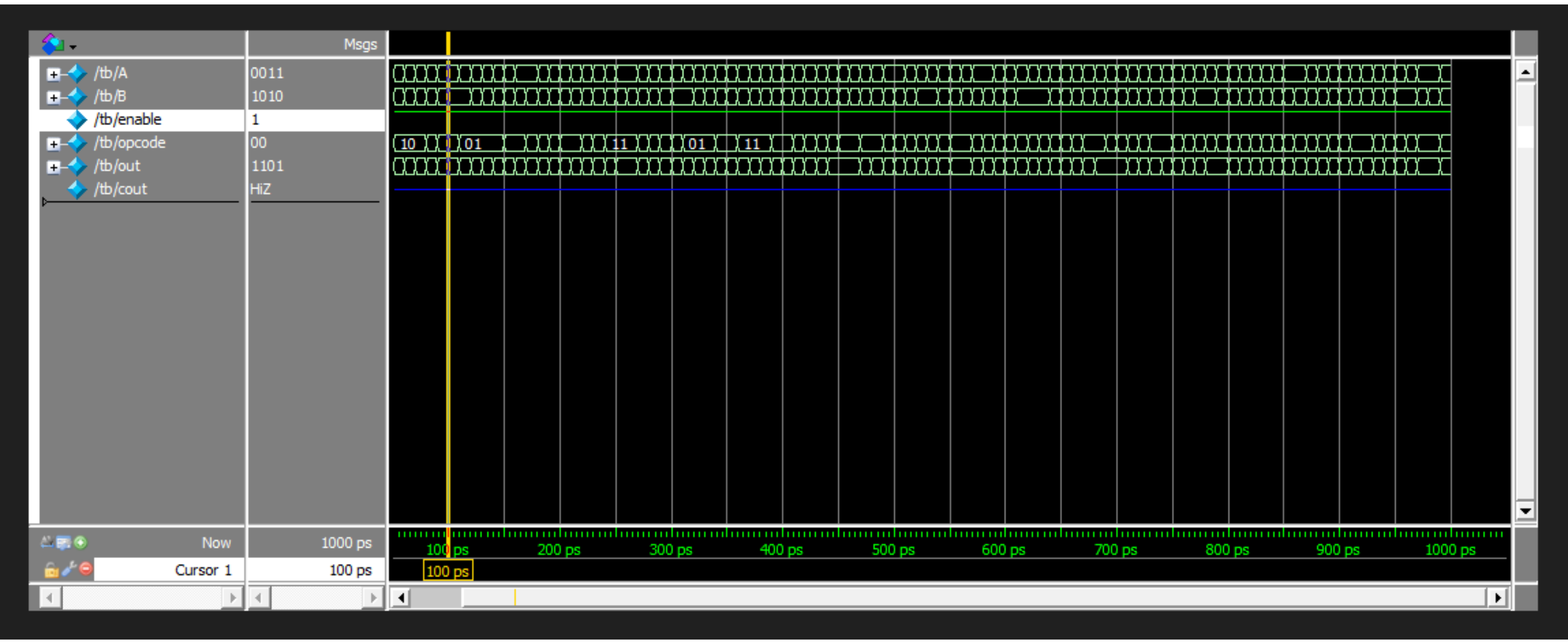
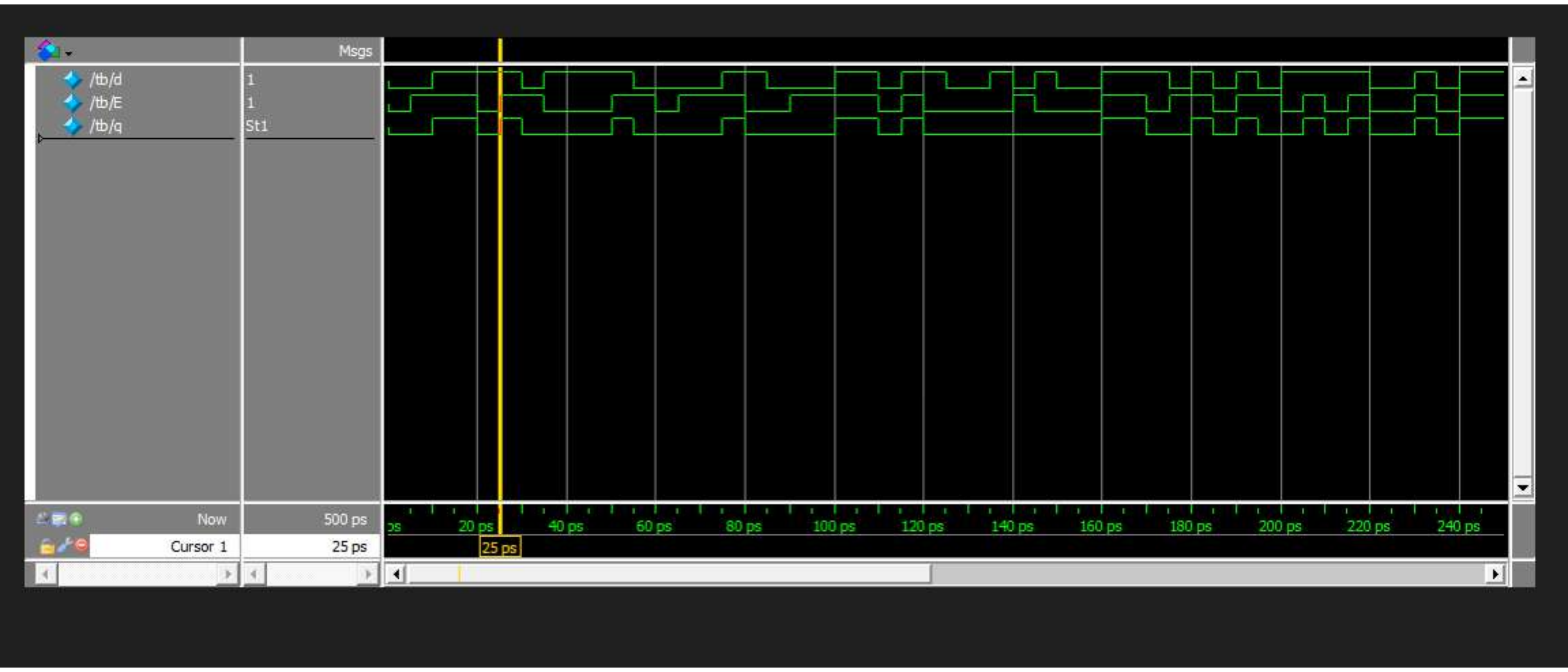
```
module tb;

 reg[3:0] A,B;
 reg enable;
 reg [1:0] opcode;
 wire [3:0] out ;
 wire cout ;

alu DUT(A,B,enable,opcode,out);


initial
begin

repeat(100)
begin
A =     $random;
B =   $random;
enable  = 1;
opcode  = $random;
#10;
end

end
endmodule
```


Msgs
/tb/A 0011
/tb/B 1010
/tb/enable 1
/tb/opcode 00
/tb/out 1101
/tb/cout HiZ
Now 1000 ps
Cursor 1 100 ps
100 ps
200 ps
300 ps
400 ps
500 ps
600 ps
700 ps
800 ps
900 ps
1000 ps

```verilog
module d_latch (
    input wire d, E,
    output reg q
);

always @(*) begin
    if (E)
        q = d;
        else q = 0;
end

endmodule
```

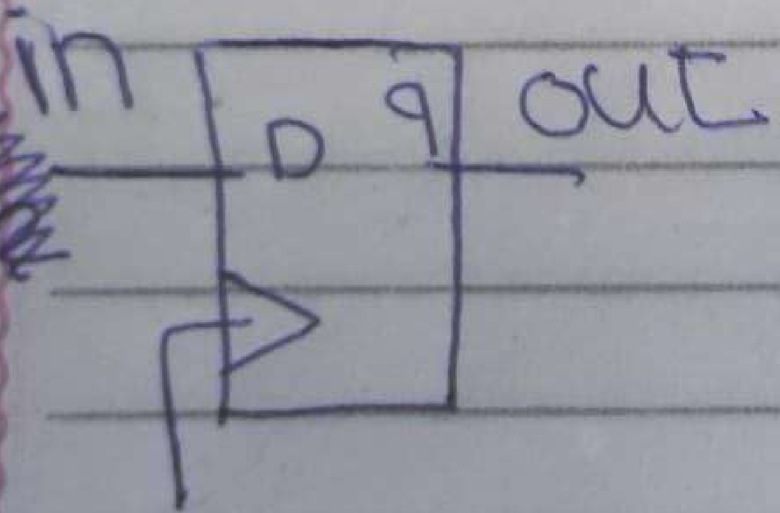
```
module tb;

reg d, E;
wire q;


d_latch DUT (d,E,q);

initial begin
    d = 0;
    E = 0;


    repeat(100) begin
        #5;
        d = $random;
        E = $random;
    end
end

endmodule
```



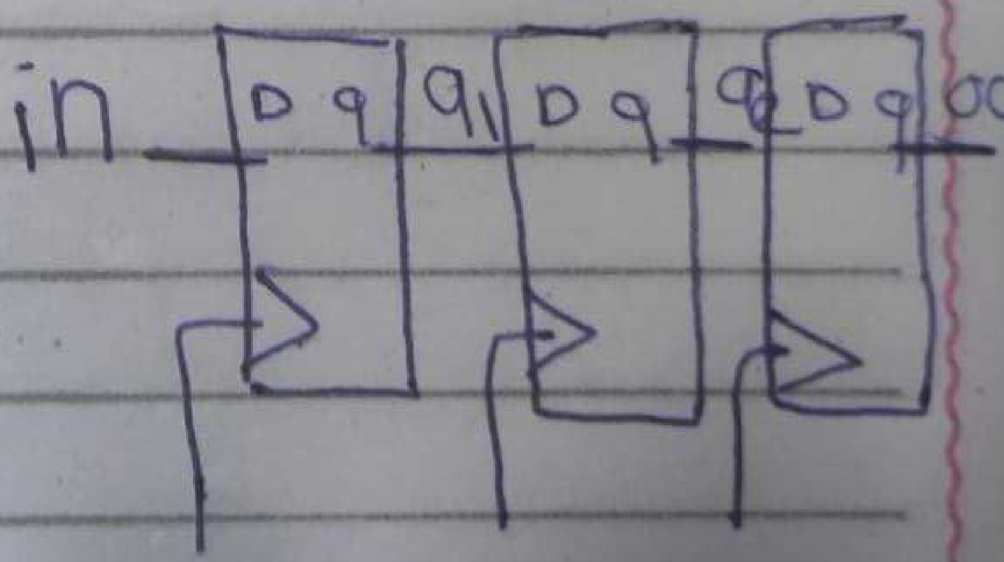
Msgs
/tb/d
1
/tb/E
1
/tb/q
St1
Now
500 ps
Cursor 1
25 ps
20 ps
40 ps
60 ps
80 ps
100 ps
120 ps
140 ps
160 ps
180 ps
200 ps
220 ps
240 ps
25 ps

```verilog
module ff #(parameter FF_TYPE = "DFF")
(
    input clk , rst_n , d ,
    output reg q , _q
);

always @ (posedge clk or !rst_n)
begin
    if(!rst_n) q<=0;
        else
            begin
                if(FF_TYPE == "DFF") q<=d;
                else if(FF_TYPE == "TFF")
                    begin
                        if(d) q<= ~q;
                        else  q<= q ;
                    end
            end
end

always @ (*) _q = ~q;

endmodule
```

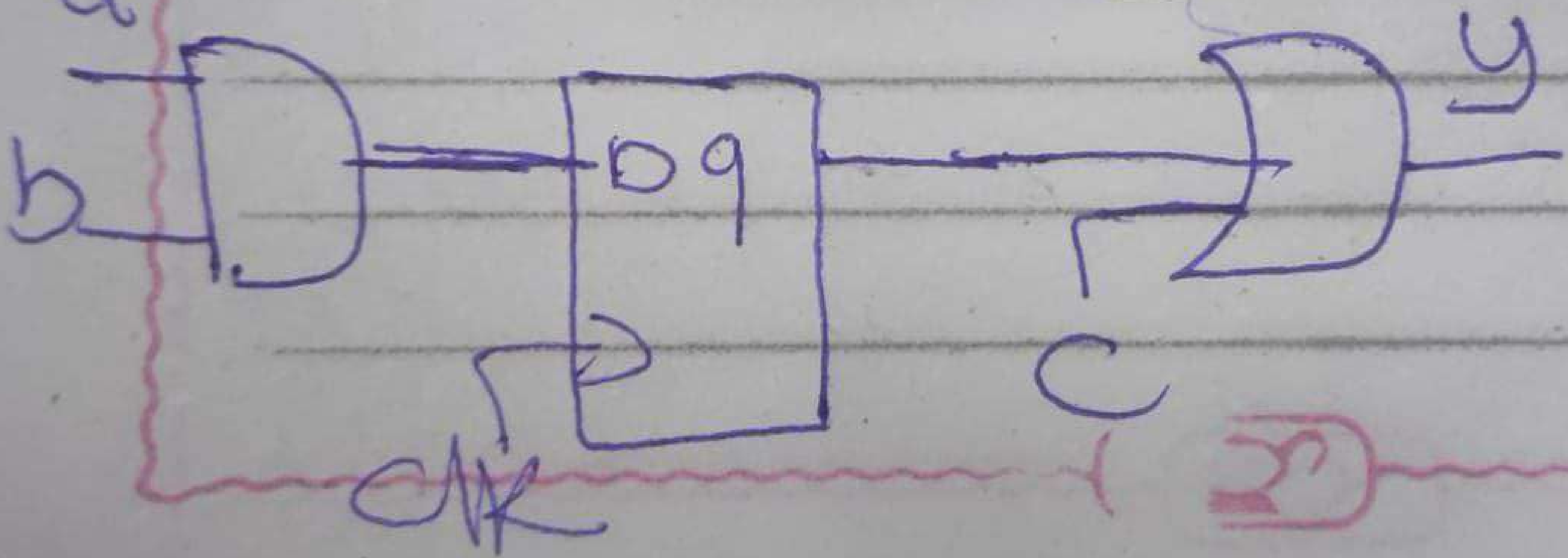
Blocking



non Blocking



Non blocking



Blocking